

它和之前 Kimi K2/Kimi Linear 的路线相比，最大的变化不是单纯“堆参数”，而是[训练体系重新设计](#)。Kimi K3 是一个 [2.8T 参数 MoE 模型 \(104B activated\)](#)，采用 KDA、Attention Residuals、Stable LatentMoE，并且训练阶段重点强化了长链路 Agent RL。
([arXiv][1])

下面重点从 [训练流程](#)、[数据](#)、[预训练](#)、[长上下文训练](#)、[RL 后训练](#)、[Agent RL 基础设施](#)几个方面拆。

1. Kimi K3 总体训练 Pipeline

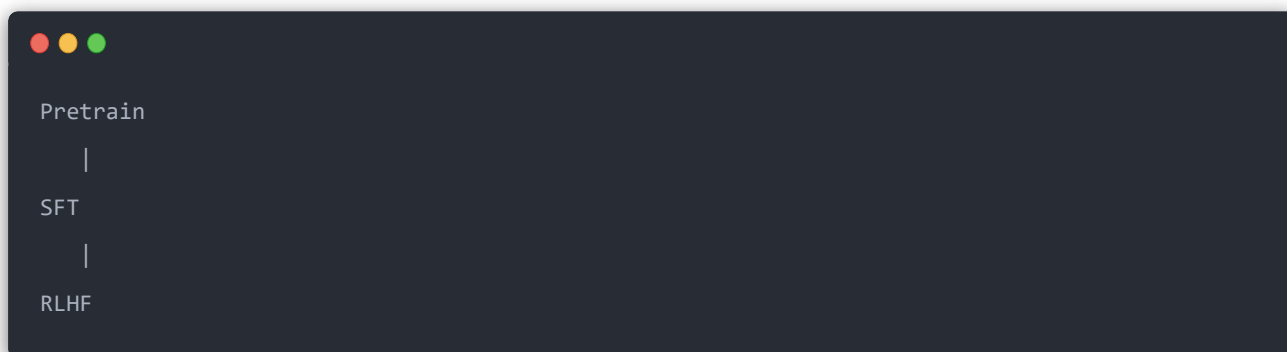
整体流程：



([arXiv][1])

和传统 LLM 最大区别：

很多模型：



Kimi K3:



更接近：

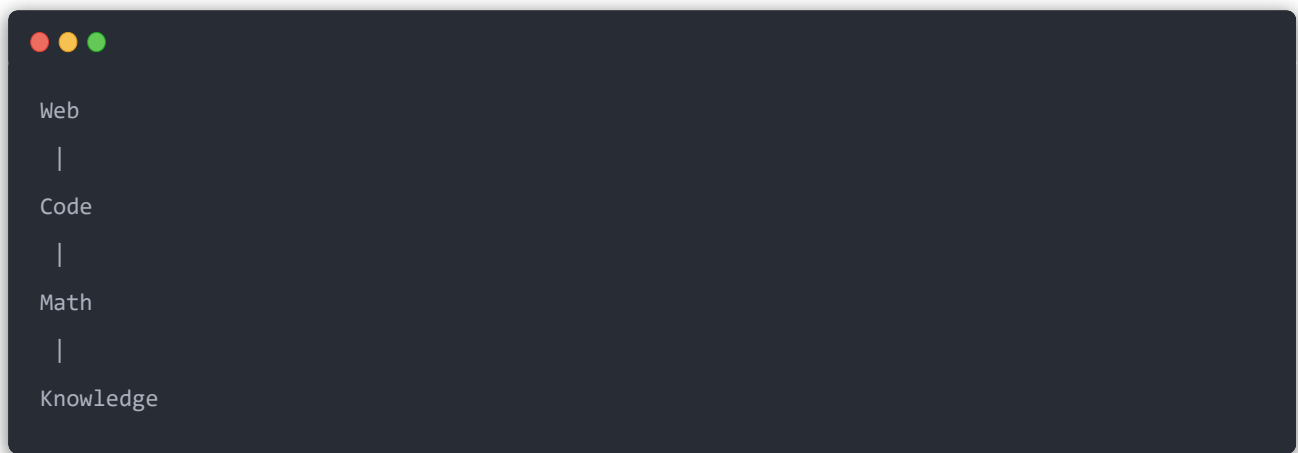
训练多个专家人格，然后合成一个统一 Agent。

2. Pre-training: 不是简单堆网页数据

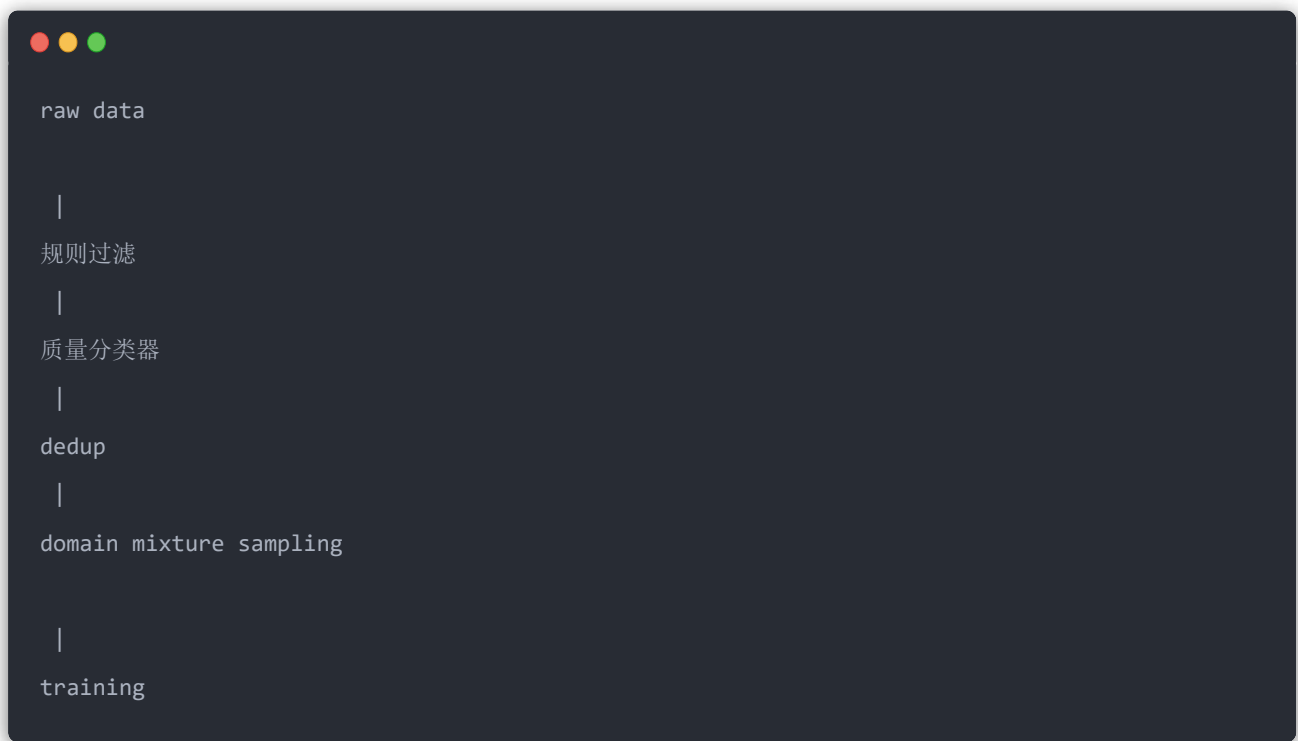
Kimi K3 的预训练数据主要分：

① Text corpus

四大类：



数据处理：



其中：

- 数学
- 知识

不是直接网页，而是进行了：

synthetic rewriting

类似：

原始：

Wikipedia:
量子力学介绍

生成：

不同表达方式
不同问题形式
不同推理路径

再训练。

原因：

减少：

memorization

增加：

reasoning diversity

3. 最大变化：架构和训练一起设计

Kimi K3 不是：

先设计模型
再训练

而是：

模型结构

+

训练方法

+

系统

一起设计

三个核心：

3.1 KDA (Kimi Delta Attention)

解决：

长上下文 Attention 成本。

传统：

$$O(n^2)$$

KDA：

维护状态：

$$S_t$$

类似：

token stream

t1

|

update memory

t2

|

update memory

t3

```
|  
read memory
```

减少：

KV cache。

3.2 Attention Residuals

解决：

深层 Transformer 信息流。

传统：

```
layer1  
+  
layer2  
+  
layer3  
...
```

K3:

```
layer history  
  
h1  
h2  
h3  
|  
attention select  
|  
current layer
```

也就是：

depth memory。

3.3 Stable LatentMoE

K3:

```
896 routed experts

每 token:

activate 16 experts
```

所以:

总参数:

2.8T

但是:

计算:

104B active。

([Hugging Face][3])

4. Long Context Training (非常关键)

Kimi K3 支持:

1 million token context

这个不是直接 pretrain 出来的。

一般流程:

```
短context pretrain

|

v
```

long context continued training

|
v

1M context

也叫：

long-context cooldown

为什么叫 cooldown?

因为训练后期：

降低学习率：



learning rate ↓

context length ↑

类似：

让模型适应：



短距离语言模型

↓

长距离记忆模型

5. RL 后训练：Kimi K3 最大创新点

Kimi K3 的 RL 不是传统：



prompt

↓

answer

↓

reward

而是：

Agentic RL

也就是：

模型真的执行任务。

例如：

任务：

修复一个 CUDA kernel

rollout:

模型

|

读代码

|

修改

|

编译

|

运行测试

|

benchmark

|

继续修改

|

最终提交

reward:

不是文本质量。

而是:



```
compile success
```

```
+
```

```
test pass
```

```
+
```

```
performance
```

```
+
```

```
task completion
```

这和你之前研究 CUDA kernel RL 非常接近。

6. 三类 RL

K3 不是一个 RL。

而是三个方向:

A. General Reasoning RL

目标:

数学:

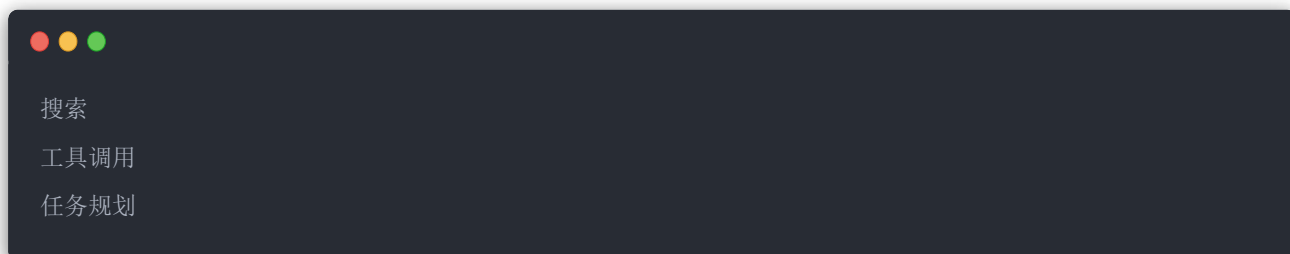


reward:

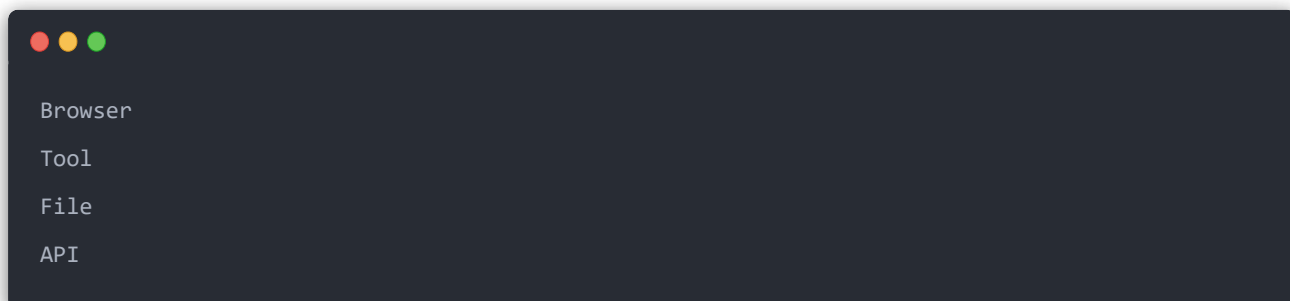
- correctness
- reasoning quality

B. General Agent RL

比如:



环境:



C. Coding Agent RL

重点:

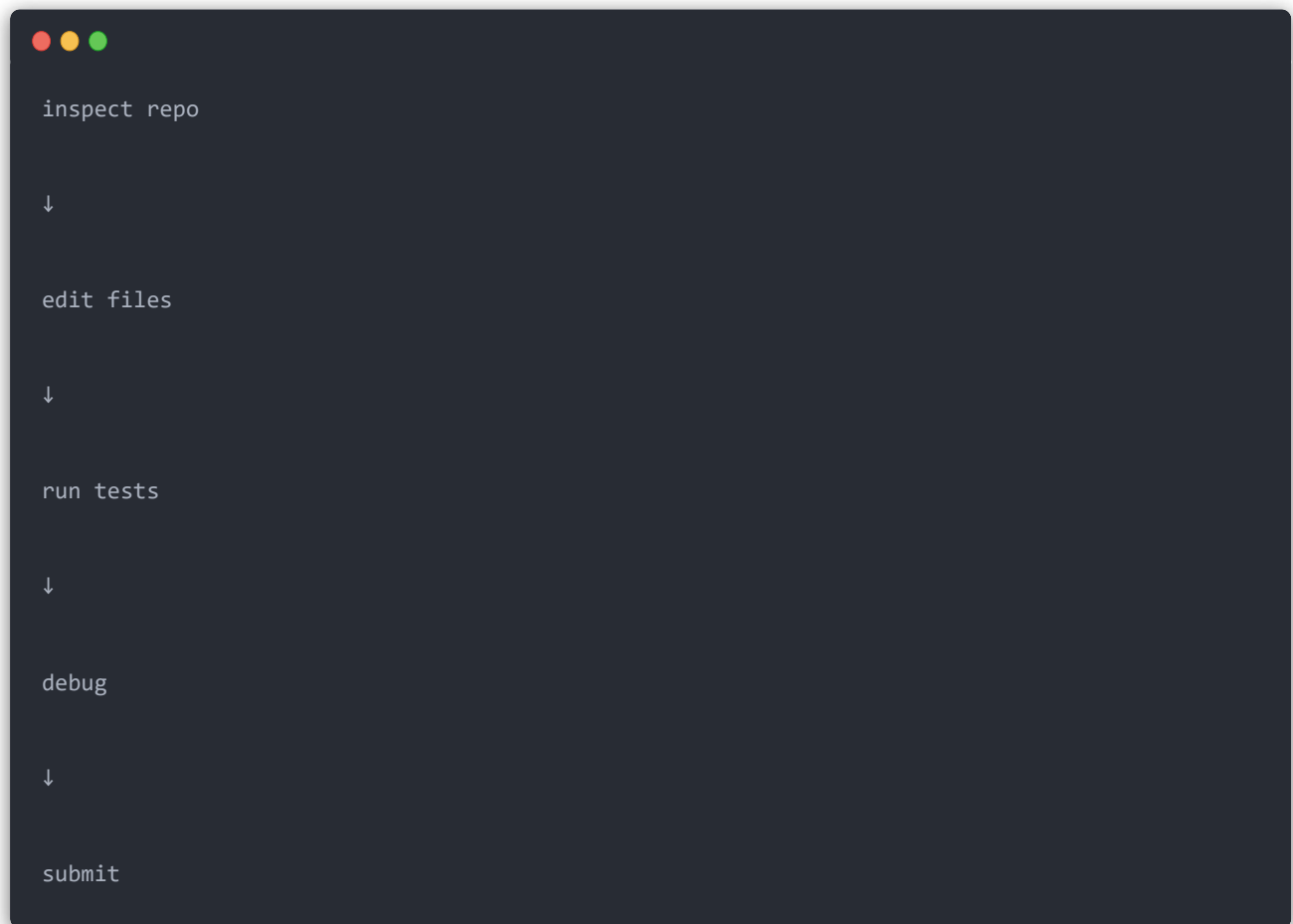
代码工程。

例如：

输入：

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text "Fix this repo" is displayed in a light gray monospace font.

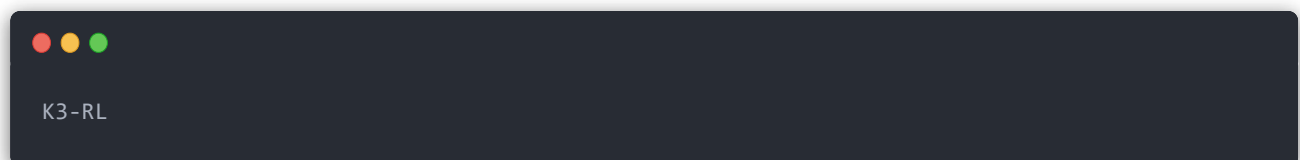
模型：

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text is displayed in a light gray monospace font, showing a sequence of commands separated by downward arrows: "inspect repo", "↓", "edit files", "↓", "run tests", "↓", "debug", "↓", "submit".

7. 多策略 RL (非常值得注意)

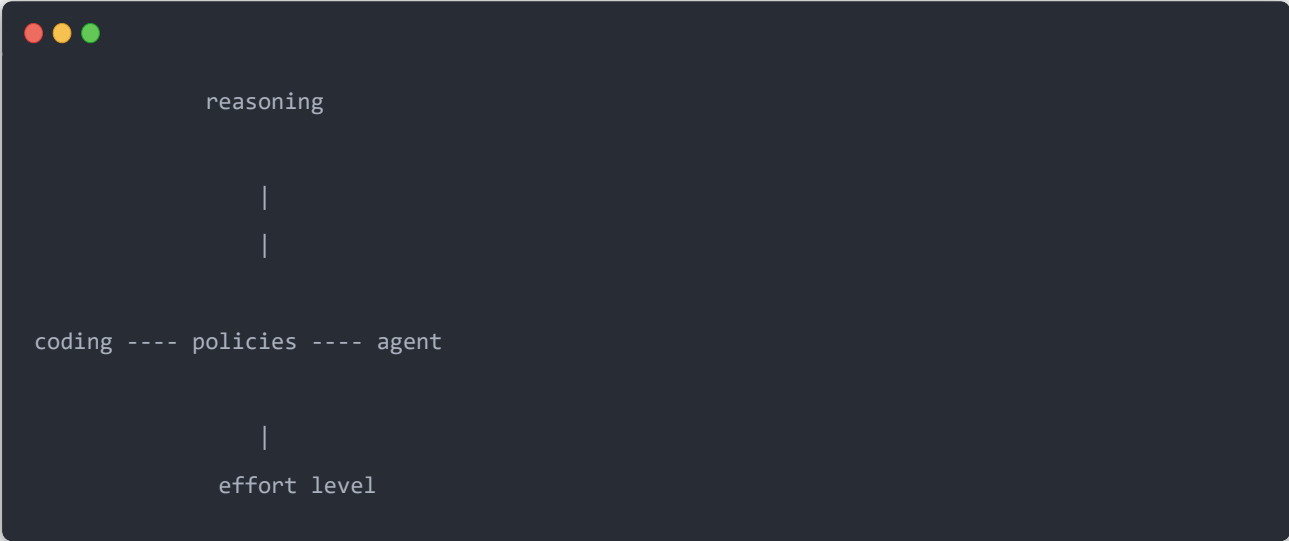
K3 不是：

一个 policy：

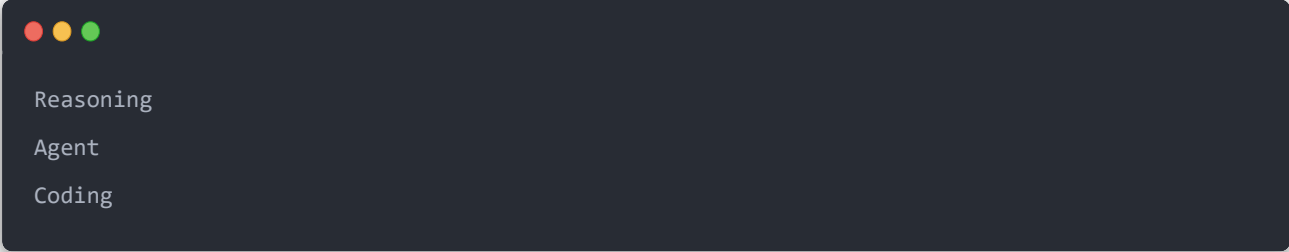
A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text "K3-RL" is displayed in a light gray monospace font.

而是：

训练多个：



三个领域：

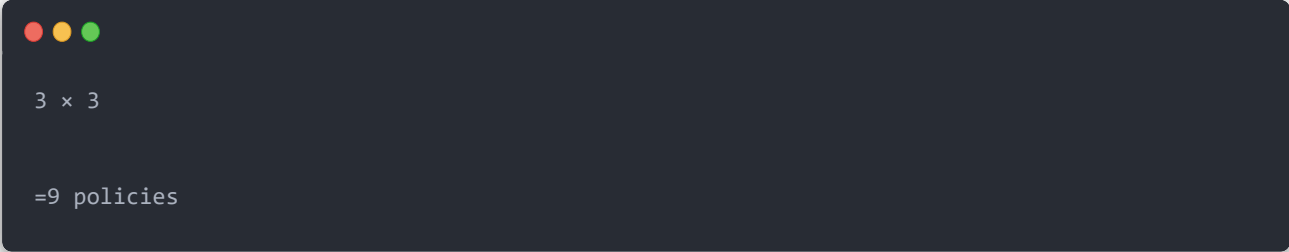


每个：

三个 reasoning effort:



总共：



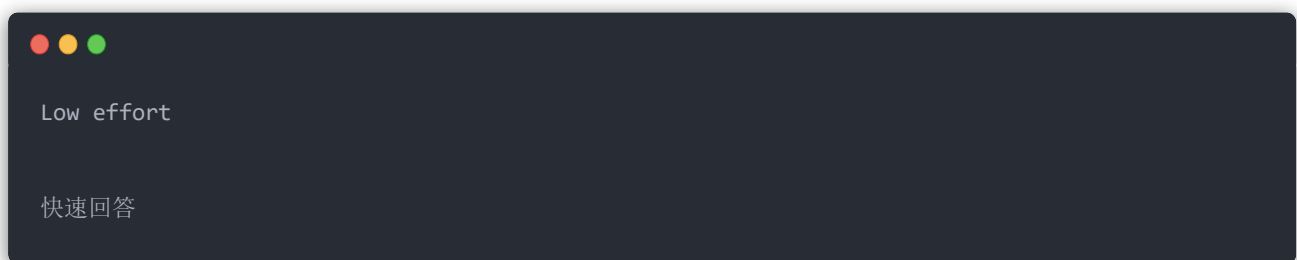
8. 为什么训练9个policy?

因为：

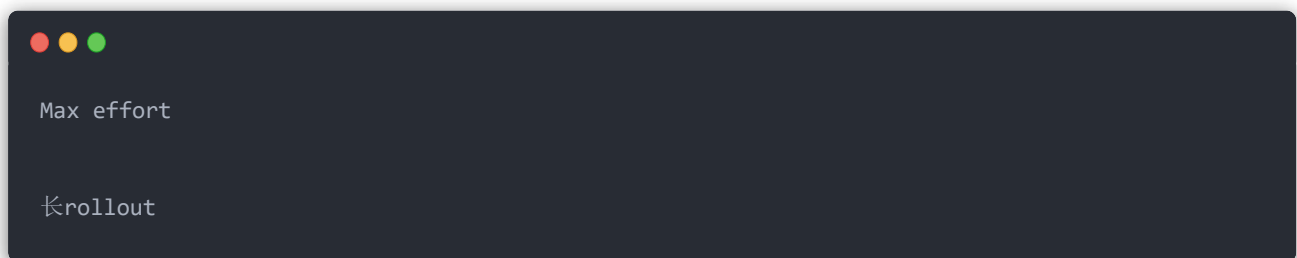
不同任务需要不同思考深度。

例如：

简单问题：

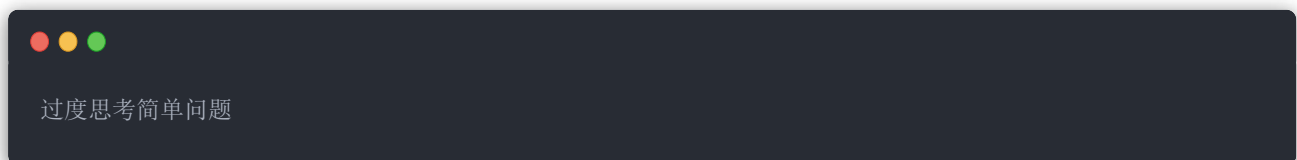


复杂代码：

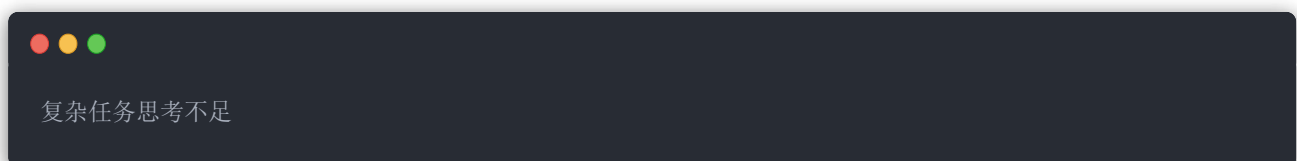


如果一个policy：

容易：



或者：



所以：

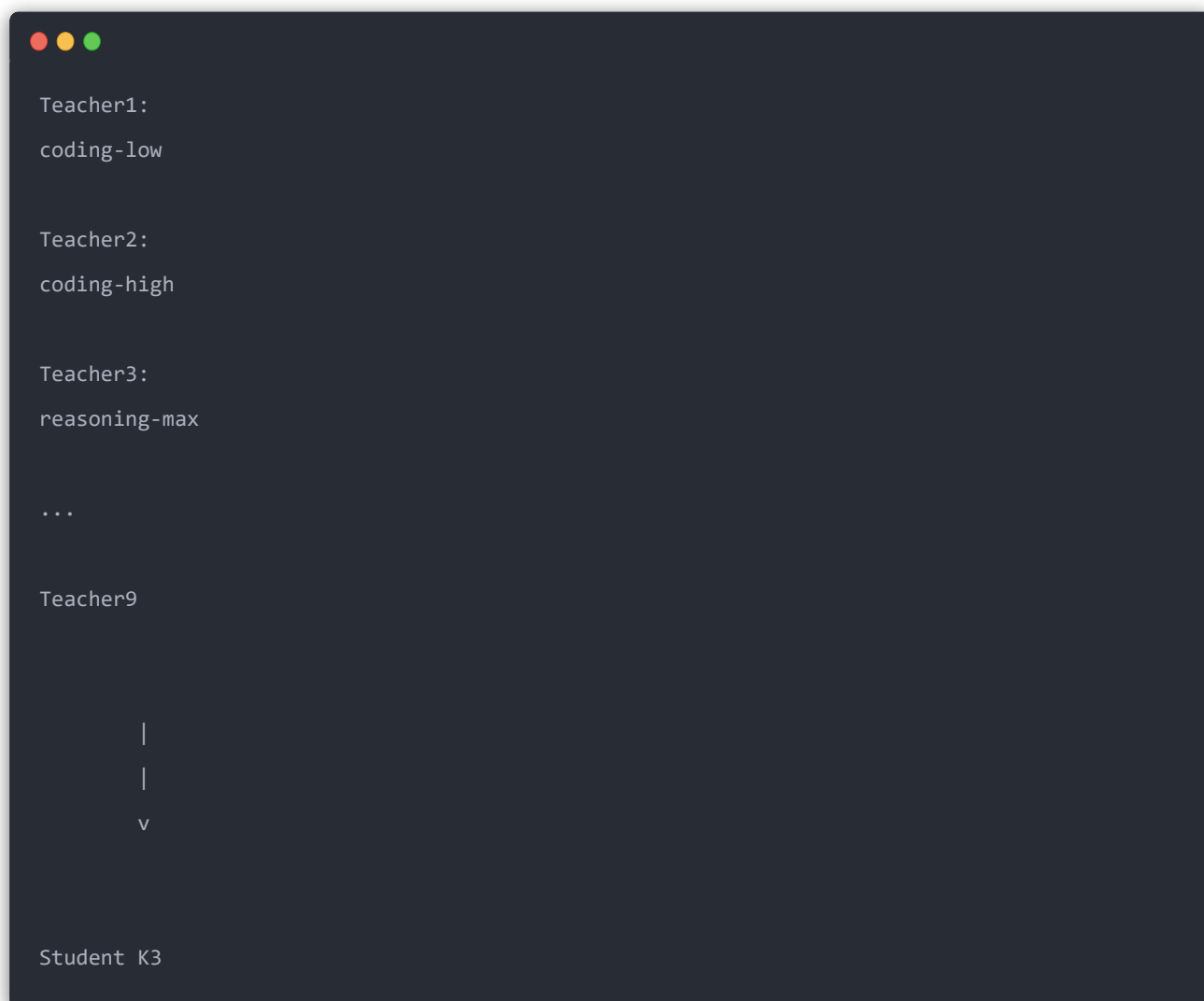
分别训练。

9. 最后怎么合成一个模型？

这里很有意思：

Multi-Teacher On-policy Distillation

流程：



学生学习：

多个 teacher 的行为分布。

类似：

$$L = \sum_i w_i KL(P_{teacher_i} || P_{student})$$

最终：

一个模型：

支持：



low/high/max reasoning

10. Agent RL 基础设施

这是 K3 和普通 RL 最大区别。

需要：

长时间 rollout。

普通：



input

output

结束

Agent:



step1:

模型调用工具

step2:

环境变化

step3:

继续

...

step100

所以他们设计：

AgentENV

特点：

- sandbox
- persistent state
- multi-turn
- resumable rollout

([Reddit][4])

11. 训练系统层面的难点

2.8T MoE 最大挑战：

Expert Parallel

896 experts:

需要：



GPU group

expert shard

routing

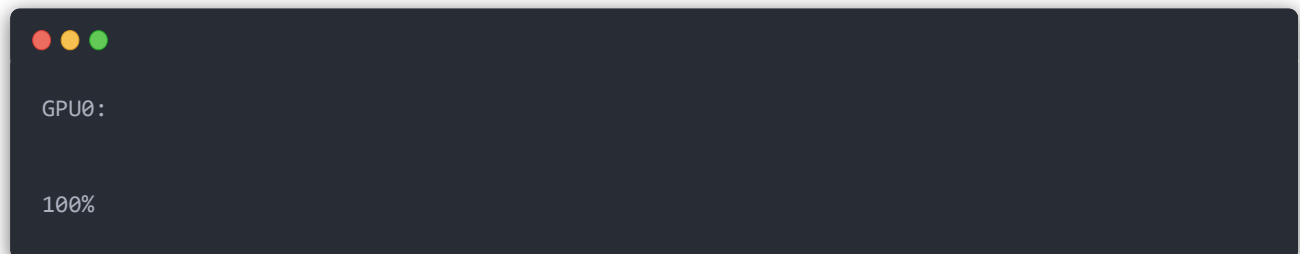
communication

关键：

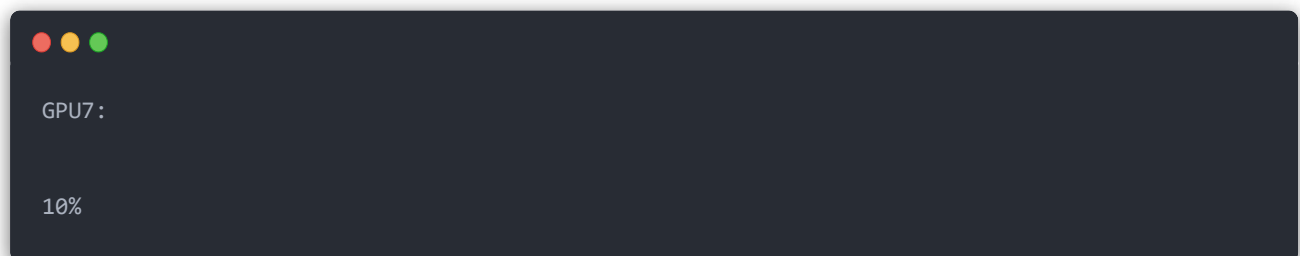
负载均衡。

否则：

某些 expert：



其他：



K3 报告强调：

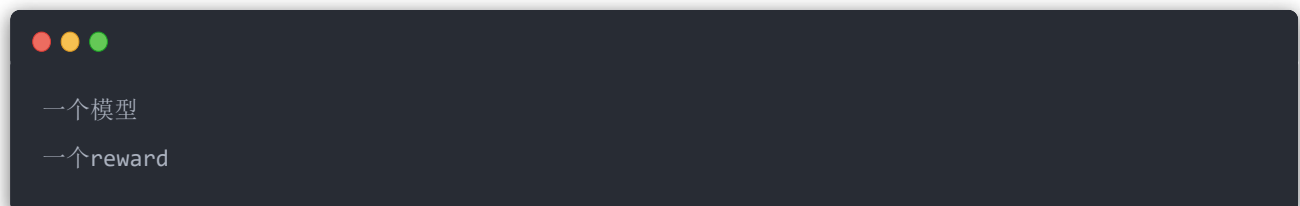
expert-parallel training + memory management 是基础设施重点。

13. 我认为 Kimi K3 最重要的三个训练创新

第一：

从单模型 RL → 多专家 RL

以前：



K3：

9个specialist policy

↓

distill

第二：

RL 从回答优化 → 行为优化

不是：

生成答案

而是：

完成任务

尤其 coding agent。

第三：

架构和训练共同设计

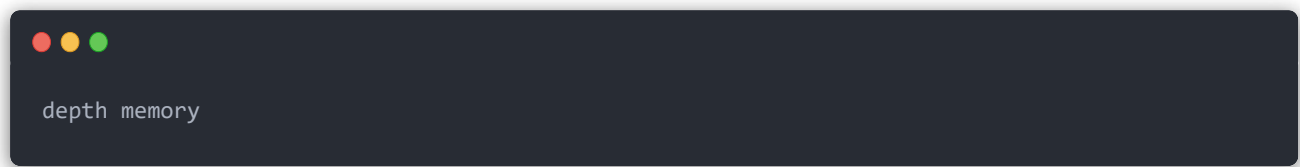
KDA：

优化：

sequence memory

AttnRes：

优化:

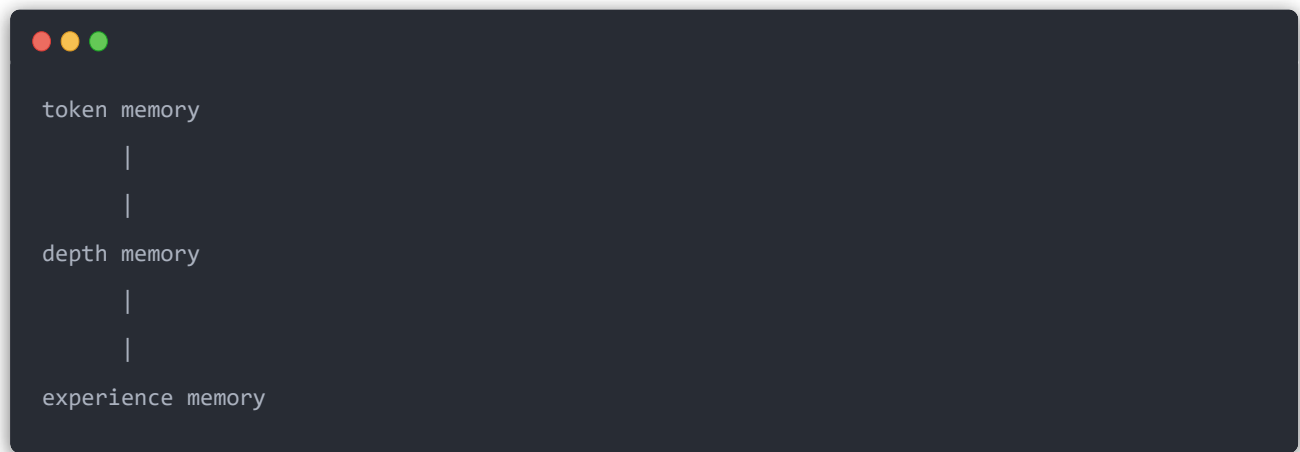


RL:

优化:



三个 memory:



这是 K3 最大的系统性设计。